

Don't Go Broke

Before the Build Starts

Where This Booklet Fits:

This is booklet 2 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- Section 1 – The Mistake Happens Before the Tool Choice
- Section 2 – Define the Job Before Defining the App
- Section 3 – Decide What Version One Really Is
- Section 4 – Create the Blueprint Before the Build
- Section 5 – Map the User Journey
- Section 6 – Define Agent Boundaries Before the Agent Starts
- Section 7 – Prepare the Handoff
- Section 8 – The Final Pre-Build Review
- Field Note Closing

Before the Build Starts

Don't Go Broke in the Age of AI

Field Note Opening

There is a moment in every AI-assisted build where the project starts costing more than it should.

It is not the moment the first credit is spent. It is not the moment the wrong platform is chosen. It is not even the moment the AI generates the wrong screen, the wrong database, or the wrong feature.

The expensive moment is earlier than all of those.

It is the moment the builder decides to start before they know what they are building.

This booklet is about that moment. It is about the gap between having an idea and being ready to hand that idea to an AI coding assistant. It is about what belongs in that gap and what happens when the gap is skipped.

If you are a first-time builder — a solo founder, a side-project creator, a student, a small business owner trying to turn an idea into something real — this booklet is for you. You do not need to be a software engineer to use it. You do not need to know how databases work or what a deployment pipeline looks like. You

need to be willing to slow down for a short time before you speed up.

The house-building analogy is overused, but it survives because it is true. No one pours a foundation before they have a blueprint. No one orders materials before they know the dimensions. No one hires a crew before they know what the crew is building.

AI coding tools are the fastest crew you have ever met. They can generate screens, databases, features, and entire applications in minutes. But speed without direction is not efficiency. It is expensive confusion.

The blueprint comes before the build.

That is the lesson of this booklet. Not because planning is glamorous. Not because blueprints are exciting. Because the alternative — handing a vague idea to a fast tool and hoping for the best — is how budgets disappear, confidence erodes, and projects stall in a pile of AI-generated code that nobody quite understands.

You are going to build something. This booklet helps you get ready.

Section 1 – The Mistake Happens Before the Tool Choice

Most first-time AI builders believe the most important decision they will make is which tool to use.

Replit or Vercel. Base44 or v0. Google Antigravity or Cursor. The forums are full of comparisons. The YouTube tutorials are full of recommendations. The temptation is strong to believe that if you just pick the right platform, the build will go smoothly.

That belief is understandable. It is also wrong.

The most expensive mistake in an AI-assisted build almost never starts with the tool. It starts with what the builder does — or does not do — before the tool is opened.

Here is the pattern we see again and again:

A builder has an idea. The idea feels clear in their head. They find an AI coding platform. They describe the idea in a prompt. The AI generates something. The something is not quite right. The builder adjusts the prompt. The AI generates again. The output drifts further. The builder adds a feature. The AI adds three more. Screens multiply. Data structures appear that nobody planned.

The builder is no longer building. The builder is reacting. And every reaction costs credits, time, and a little more confidence.

The tool did not cause this. The tool did exactly what it was asked to do. The problem is that what it was asked to do was vague, and vague instructions given to a fast tool produce expensive confusion faster than slow tools ever could.

This is the core warning of this booklet: the mistake happens before the tool choice. The mistake is building without a clear definition of what you are building.

A missing definition is not obvious. It does not announce itself. It feels like momentum. The builder feels productive because something is being generated. But productivity without direction is just motion, and motion without a target is how projects burn through their budget before they have anything worth keeping.

The tradeoff is real: spending time on definition feels like it slows you down. It feels like procrastination. It feels like the kind of planning that "real" builders skip because they are too busy shipping. But the builders who skip definition are the ones who end up rebuilding the same feature four times, explaining to the AI what they actually meant, and staring at a credit usage dashboard wondering where the money went.

The tool cannot save you from a missing definition. No platform can. The builder who arrives at any AI tool with a clear definition of the problem, the user, and the intended outcome will

outperform the builder who arrives with excitement and a vague idea, regardless of which platform either one chooses.

The first step is not picking a tool.

The first step is knowing what you need the tool to build.

Section 2 – Define the Job Before Defining the App

Before you think about screens, features, databases, or AI prompts, you need to answer a question that sounds simple and is not:

What job is this thing supposed to do?

Not "what does it look like." Not "what features does it have." Not "what technology does it use." The job. The actual work the product performs for the person who uses it.

Most first-time builders skip this question because they think they already know the answer. They have an idea. The idea feels clear. But there is a significant difference between having an idea and having a defined job, and that difference is where expensive mistakes begin.

An idea sounds like: "I want to build a task manager."

A job sounds like: "A solo freelancer needs to see which client tasks are due this week so they can prioritize without missing deadlines."

The first version is a category. The second version is a job. The difference matters because the category invites unlimited features — tags, filters, calendars, collaboration, notifications, integrations, dashboards, reports — while the job constrains what the first version actually needs to do.

When you hand an AI coding assistant a category, the AI will build a category. It will generate a full-featured task manager because that is what task managers look like in its training data. You will get a project with twenty screens, a database with twelve tables, and features you never asked for. Then you will spend your budget trimming the AI's interpretation back toward what you actually needed.

When you hand an AI coding assistant a job, the AI has a target. It knows who the user is. It knows what the user needs to accomplish. It knows what success looks like. The output will be smaller, more focused, and far easier to review.

Defining the job is not a creative writing exercise. It is a practical act of constraint. You are drawing a boundary around the problem so that everything inside the boundary is essential and everything outside the boundary can wait.

Here is a practical way to write the job: complete these three statements in plain language.

The user is: (who specifically will use this)

The problem is: (what specific difficulty the user faces)

The job is: (what the product must do to solve that specific problem)

Write each answer in one or two sentences. If you need a paragraph, the job is not defined yet — it is still an idea trying to become a job.

The warning here is subtle. Many builders define the job in their head but never write it down. They believe they will remember. They believe the definition is obvious. But when the AI starts generating, the builder's memory of the job competes with the AI's output, and the AI's output is visual, immediate, and persuasive. A screen that looks good can make the builder forget that it does not match the original intent. A written job statement cannot be overwritten by a pretty interface.

The tradeoff: writing the job takes fifteen minutes. Skipping the job and correcting the AI later takes hours, costs credits, and often produces a project that solves a slightly different problem than the one the builder actually cared about.

Fifteen minutes of definition before building. That is the price. The alternative is paying ten times that in rework after building.

Section 3 – Decide What Version One Really Is

Every builder wants to build the dream version.

The dream version has all the features. It handles every edge case. It looks polished. It impresses everyone. The dream version is the product the builder imagines when they close their eyes and picture the finished result.

The dream version is also dangerous.

Not because it is bad. Because it is big. And big things built by first-time builders with AI tools have a specific failure mode: they start ambitious, generate fast, accumulate complexity, become hard to review, exceed the budget, and collapse under the weight of features that were added before the foundation was solid.

Version one is not the dream version. Version one is the proof that the job is real and the solution works.

This is one of the hardest lessons for a first-time builder to accept. The AI makes it feel possible to build everything at once. The generation is fast. The screens appear quickly. The features seem to assemble themselves. But speed of generation is not the same as quality of design, and a product that was generated quickly

from a vague prompt is not the same as a product that was built carefully from a clear definition.

Version one should do one thing well. It should prove that the core job works. It should be small enough to review completely, affordable enough to finish, and simple enough that you can tell whether the AI built what you asked for.

Here is a practical way to think about scope. Take every feature you have imagined for the product and sort it into three groups:

Must work. These are the features without which the core job cannot be performed. If the product is a task tracker for freelancers, the ability to add a task and see tasks due this week is a "must work." Without it, the product does not do its job.

Can wait. These are features that would make the product better but are not required to prove the job works. Notifications, export to PDF, color coding, integrations with other tools — useful, but not essential for version one.

Not yet. These are features that belong to the dream version. Team collaboration, analytics dashboards, mobile apps, third-party API connections. They may be important eventually. They are dangerous now because they multiply the number of things the AI builds, the amount of code you must review, the number of ways things can break, and the cost of the entire project.

The tradeoff is emotional. Cutting features feels like cutting ambition. It feels like settling. But the builder who ships a small, working version one and then expands it is in a fundamentally different position than the builder who attempts the dream version, runs out of budget, and has nothing to show for it.

AI tools are especially dangerous here because they do not push back on scope. If you ask for twenty features, the AI will attempt twenty features. It will not tell you that the database design is getting complex, that the user flow has gaps, or that the budget cannot support this much generation. The AI builds what you ask for. If you ask for too much, you get too much — and "too much" is not free.

The discipline of version one is the discipline of saying: "I will prove the job works before I make the product bigger." That sentence protects your budget, your timeline, and your ability to understand what the AI has built.

Decide what version one really is before you open the AI tool. Write it down. Make the list short. If the list is not short, you are building the dream version, and the dream version is the one most likely to cost more than you planned.

Section 4 – Create the Blueprint Before the Build

By now you have a defined job and a constrained version one. You know what the product is supposed to do, who it serves, and what it must not try to do yet.

The next step is not opening the AI tool.

The next step is creating the blueprint.

A blueprint is a simple document that maps what the product looks like, how it works, and what it contains. It is not a technical specification. It is not a professional wireframe. It is not a software architecture diagram. It is a plain-language description of the product that is clear enough for another person — or an AI — to understand what you intend.

The blueprint matters because it becomes the contract between you and the AI. When the AI generates the first version of your product, the blueprint is how you evaluate whether the output is correct. Without a blueprint, you are reviewing the AI's work against the idea in your head, and the idea in your head is a moving target. With a blueprint, you are reviewing the AI's work

against a written document, and a written document does not change every time you see a shiny new screen.

Here is what a practical blueprint should include:

Purpose. One sentence. What does this product do? This is a compressed version of the job you defined earlier.

User. One sentence. Who uses this product? Not "everyone." A specific person in a specific situation.

Core flow. The main path a user takes through the product to accomplish the job. This does not need to be a formal diagram. It can be a short sequence of five to seven steps described in plain sentences.

Required screens. What screens or views does the product need to support the core flow? List them. If you have more than eight screens in version one, you are probably building more than version one.

Required data. What information does the product store? This does not need to be a database schema. It needs to be a plain-language list of what the product remembers.

What is excluded. This is as important as what is included. Write down what the AI must not build in this version. No notifications. No team features. No reporting dashboard. No integrations. This list prevents the AI from expanding the project beyond what you can afford and review.

The warning about blueprints is this: they feel unnecessary. The builder knows what they want. The AI is fast. Writing it down feels like busywork. But the builders who skip the blueprint are the ones who end up with AI-generated products that have extra screens, extra features, extra database tables, and extra complexity that nobody asked for and everybody has to pay for.

The blueprint is not bureaucracy. It is protection. It takes thirty minutes to write. It saves hours of rework, dozens of credits, and the specific kind of frustration that comes from looking at an AI-generated product and thinking, "This is not what I meant, but I cannot explain why because I never wrote down what I meant."

Write the blueprint. Keep it to one page. Make it plain. Then use it as the measuring stick for everything the AI builds.

Section 5 – Map the User Journey

The blueprint tells you what the product contains. The user journey tells you how the product works from the user's perspective.

These are different things, and conflating them is a common source of expensive rework.

A product can have all the right screens and still be confusing to use. A product can have all the right data and still force the user through an illogical sequence. A product can have every feature on the "must work" list and still fail because nobody mapped how the user moves from start to finish.

A user journey is the step-by-step path a real person takes through the product to accomplish the job. It starts when the user arrives and ends when the job is done.

Here is what a user journey looks like in practice: The user opens the app for the first time. They see a simple sign-up screen. They create an account. They land on an empty screen with a clear prompt to take their first action. They complete it. They see the result appear. They repeat the action a few more times. They check their progress. They finish and close the app knowing their work is saved.

That is a user journey. It is not a feature list. It is not a screen list. It is a story about a person using the product. And that story reveals things the blueprint alone does not.

It reveals where the user might get stuck. What happens after sign-up? Is there an empty state, or does the user land on a blank screen with no guidance? What happens if the user skips a required field? Is the next step obvious?

It reveals missing screens. The blueprint might list a main view and an input form, but the user journey reveals that you also need a first-use welcome state, a confirmation after saving, and a way to see completed items separately.

It reveals unnecessary screens. If the blueprint includes a screen that the user journey never visits during the core job, that screen might belong in "can wait" instead of version one.

The tradeoff of mapping the user journey is time. It takes twenty to thirty minutes to write through the journey carefully. That time investment feels expensive when the AI can generate screens in seconds. But the screens the AI generates without a user journey are screens arranged by the AI's best guess, not by the user's actual needs. Rearranging AI-generated screens to match a journey you discover after the build is far more expensive than mapping the journey before the build and giving the AI the map.

There is a specific danger with AI-generated products that user journey mapping prevents. AI tools are trained on patterns from

existing applications. When you describe your product by category, the AI will generate a product that looks like every similar product in its training data — complete with features, screens, and flows that your user does not need. The user journey is your defense against this. It defines what the user actually does, which prevents the AI from building what products in that category generally do.

Map the journey before you build. Write it as a story, not a diagram. Keep it focused on the core job. If the journey has more than ten steps, version one might be too big.

Section 6 – Define Agent Boundaries Before the Agent Starts

AI coding agents are powerful, fast, and helpful. They are also unbounded by default.

When you hand an AI agent a description of your product, the agent will build what it understands from that description. If the description is vague, the agent's interpretation will fill in the gaps. The agent will make decisions about architecture, database design, screen layout, feature scope, third-party integrations, and deployment configuration. It will make these decisions quickly, confidently, and without asking permission.

This is not a flaw in the AI. This is how AI agents work. They generate the most complete version of what they think you asked for. The problem is that "the most complete version" is often much larger, more complex, and more expensive than what you needed.

Agent boundaries are the rules you set before the agent starts building. They define what the agent may do, what it must not do, and where it must stop and wait for your review.

Without boundaries, the agent is likely to:

Add third-party services you did not request. Authentication providers, payment processors, analytics platforms, email services — the agent knows these are common in applications, and it will include them unless you say otherwise.

Create more database tables than you defined. Your blueprint says two or three data types. The agent might create seven or eight, adding categories, tags, activity logs, notifications, and user preferences. Each additional table adds complexity, and complexity adds cost.

Build screens you did not plan. The blueprint says four screens. The agent might build eight, including a settings page, an admin panel, a reporting dashboard, and an onboarding wizard. Each screen must be reviewed, tested, and maintained.

Choose infrastructure that exceeds your needs. The agent might configure a production-grade database, a caching layer, a load balancer, and an automated deployment pipeline for a project that needs a simple lightweight database and a single server.

The boundaries you set should cover at least these areas:

What the agent may build. Only the screens, features, and data structures listed in the blueprint. Nothing additional without explicit approval.

What the agent must not build. No third-party integrations. No features from the "can wait" or "not yet" lists. No

infrastructure beyond what is needed for a working prototype.

Where the agent must stop. After generating the initial version, the agent should stop and present the output for review. It should not iterate, improve, refactor, or "enhance" without being asked.

What the agent must not decide. Technology choices, hosting providers, database engines, and authentication methods should be specified by you or discussed before the agent acts. The agent should not make these decisions silently.

The tradeoff of setting boundaries is that it requires you to think through the build before the build happens. That takes time. But the time you spend setting boundaries is dramatically less than the time you will spend later removing features, undoing integrations, simplifying architecture, and recalculating costs that grew because the agent did what agents do — it built the biggest, most complete version it could imagine.

Setting boundaries is not about distrusting the AI. It is about respecting the fact that the AI does not know your budget, your timeline, your skill level, or your tolerance for complexity. You know those things. The boundaries are how you communicate them.

Write the boundaries down. Include them in the prompt. Make them explicit. An agent without boundaries will build freely. An agent with boundaries will build what you asked for.

Section 7 – Prepare the Handoff

Everything you have done so far — defining the job, constraining version one, creating the blueprint, mapping the user journey, setting agent boundaries — has been preparation. Now it is time to package that preparation into something the AI can use.

This is the handoff. And the handoff is where all the careful thinking either pays off or falls apart.

The lesson of the handoff is deceptively simple: the AI can only build safely what you have clearly communicated. If your handoff is complete, the AI has a defined target and clear constraints. If your handoff is incomplete, the AI will fill in the gaps with its own assumptions, and its assumptions will cost you money.

A good handoff is not a long document. It is not a formal specification. It is a structured collection of the decisions you have already made, organized so the AI can act on them without guessing.

Here is what the handoff should contain:

The job statement. The one or two sentences that define who the user is, what problem they face, and what the product does

about it. This gives the AI context for every decision it makes during the build.

The version one scope. The "must work" features only. The "can wait" and "not yet" lists should be mentioned as explicit exclusions so the AI knows what not to build.

The blueprint. The purpose, user, core flow, required screens, required data, and exclusion list. This is the structural contract. The AI should build to this specification.

The user journey. The step-by-step narrative that tells the AI how the screens connect, what the user does on each screen, and what "done" looks like.

The agent boundaries. The rules that define what the agent may build, must not build, must not decide, and where it must stop for review.

When these elements are packaged together, the AI receives a complete picture. It knows the job. It knows the scope. It knows the structure. It knows the flow. It knows the constraints. It can build from definition instead of from assumption.

When any of these elements is missing, the AI must guess. And guessing is expensive.

A missing job statement means the AI will invent a purpose based on the app category. A missing scope means the AI will build the dream version. A missing blueprint means the AI will decide the

screens and data model for you. A missing user journey means the AI will arrange screens in the order that seems logical to a language model, not to your user. Missing boundaries mean the AI will expand freely.

Each guess adds work. Each piece of added work costs credits. Each credit is money, time, and attention that could have been spent on the actual product instead of correcting the AI's interpretation of a vague request.

The handoff is also your baseline for review. When the AI returns its first output, you compare the output against the handoff. Does the output match the blueprint? Does it follow the user journey? Does it respect the boundaries? Does it solve the job? If you do not have a handoff document, you are reviewing the AI's output against your memory, and memory is unreliable, especially when a polished AI-generated interface is persuading you that what you see is what you wanted.

Build Receipt

Once the handoff is assembled, create a short Build Receipt — a single-page reference that captures the essentials in compressed form. The Build Receipt is not the handoff itself. It is a snapshot you can return to quickly during review and future iterations.

The Build Receipt should record:

- **Problem definition:** The one-sentence job statement.
- **User persona:** Who the user is and what situation they are in.
- **Success criteria:** How you will know the product is working.
- **Blueprint snapshot:** The core flow and required screens, listed briefly.
- **Agent boundaries:** The key constraints, summarized.

Keep the Build Receipt to one page. It serves as a quick-reference audit tool — something you glance at before approving the AI's next round of changes. It is not a lesson. It is not a framework. It is a receipt that proves you made the decisions before the AI started building.

Prepare the handoff before you open the AI tool. Review it once. Then hand it over. The thirty minutes you spend assembling the handoff will save you the most expensive hours of the entire project: the hours spent correcting a build that started without one.

Section 8 – The Final Pre-Build Review

You have the job defined. You have version one constrained. You have the blueprint written. You have the user journey mapped. You have the agent boundaries set. You have the handoff assembled.

Before you open the AI tool, stop one more time.

This final review is not a checklist. It is a set of questions you ask yourself to make sure the preparation is genuinely complete, not just technically present.

The difference matters. A builder can have a blueprint that says "task tracker" and a user journey that says "user adds task." Those artifacts exist, but they are not useful. They are placeholders wearing the disguise of preparation. The final review is where you catch that.

Ask yourself:

Can I explain the job to a stranger in two sentences? If you cannot, the job is not defined yet. It is still a feeling, not a statement. Go back and write it until it is clear enough to say out loud.

If I removed every feature except the "must work" list, would the product still do its job? If not, something on the "must work" list is missing, or something that should be on the "can wait" list has crept in. Go back and re-sort.

Could someone who has never spoken to me read the blueprint and understand what the product is? If not, the blueprint is still in your head. Write what you are seeing until it is on the page.

Does the user journey start from nothing and end with the job complete? If there is a gap — a step where you are not sure what happens — the journey is not finished. Fill in the gap before the AI fills it in for you.

Would I be comfortable if the AI built exactly what the handoff says and nothing more? If the answer is "I would want it to add a few things," those things need to be in the handoff or on the "can wait" list. The AI should not be expected to read your mind.

Do the boundaries actually prevent the AI from expanding the project? If the boundaries are vague — "keep it simple" or "don't overdo it" — they are not boundaries. They are wishes. Write boundaries the AI can follow literally.

This review should take ten minutes. If it takes longer, it is because one of the earlier steps is incomplete, and that is exactly the kind of problem this review is designed to catch.

The temptation at this stage is powerful. You have been preparing. You want to build. The AI is right there, waiting. Every minute spent reviewing feels like a minute wasted. But the builders who rush through the final review are the ones who discover gaps mid-build, when fixing them costs five times more than finding them now.

There is one more question worth asking, and it is the most important one:

If the AI builds exactly what I have described, will I be able to tell whether the result is correct?

If the answer is no, the definition is not complete enough for safe building. The purpose of all this preparation is not to create a perfect plan. It is to create a plan clear enough that you — the builder, the person spending the money and the time — can look at the AI's output and say with confidence: "This matches what I asked for" or "This does not."

That is what readiness looks like. Not perfection. Clarity.

When you can answer yes to that final question, you are ready.

Pre-Build Readiness Checklist

The questions above are the lesson. This checklist is only a final safety pass — a quick way to confirm that the decisions you made in each section are documented and ready to hand over. Do not

use it as a substitute for the thinking. Use it as a confirmation that the thinking is done.

- ☐ Problem defined: you can state the specific problem in one or two sentences.
- ☐ User defined: you can describe who the user is and what situation they are in.
- ☐ Success criteria established: you know how to tell whether the product is working.
- ☐ Version one boundary set: "must work," "can wait," and "not yet" are separated and written down.
- ☐ Blueprint complete: purpose, user, core flow, required screens, required data, and exclusions are documented on one page.
- ☐ User journey mapped: the step-by-step path from arrival to job completion is written as a narrative.
- ☐ Agent boundaries set: what the AI may build, must not build, must not decide, and where it must stop are explicit.
- ☐ Handoff package assembled: job statement, scope, blueprint, journey, and boundaries are collected in one place.
- ☐ Build Receipt created: a one-page snapshot of the key decisions exists for later review.
- ☐ Readiness confirmed: if the AI builds exactly what you described, you will be able to tell whether the result is correct.

A checked box is not readiness. Readiness is the clarity behind the check. If any box is checked but the decision behind it is vague, go back to the relevant section and make it concrete before proceeding.

Open the tool. Hand over the blueprint. Let the AI build what you designed.

Field Note Closing

The lesson of this booklet is not complicated.

The expensive mistake is not choosing the wrong platform. It is not picking the wrong AI model. It is not writing a bad prompt or using the wrong programming language.

The expensive mistake is starting before you know what you are building.

That mistake hides behind excitement. It hides behind the speed of AI tools. It hides behind the feeling that planning is procrastination and building is progress. But building without definition is not progress. It is motion. And motion without a target is the fastest way to spend your budget, lose your confidence, and end up with a product you cannot explain, review, or finish.

The preparation described in this booklet is not long. A job statement takes fifteen minutes. Version one constraints take twenty. A blueprint takes thirty. A user journey takes twenty. Agent boundaries take fifteen. The handoff takes fifteen. The final review takes ten.

That is roughly two hours. Two hours of thinking before building.

The alternative — skipping the thinking and going straight to the AI — saves those two hours up front. But it costs them back many times over in rework, in wasted credits, in features nobody asked for, in screens that do not connect, in databases that do not match the product, and in the slow, frustrating realization that the project has drifted so far from the original idea that starting over might be cheaper than fixing what exists.

Two hours of preparation. Or ten hours of correction. That is the tradeoff this booklet is about.

You are going to build something. You have an idea worth pursuing. The AI tool you choose will be fast, capable, and eager to generate.

Before it starts generating, make sure it knows what to build.

The blueprint comes before the build. Always.

The Next Step

You are ready for the next phase. Proceed to booklet: **Before the Agent Starts.**